

Guía de Aprendizaje — Caso Ferreycorp paso a paso

Para qué sirve este documento: explicarte, desde cero y con detalle, **qué construimos, por qué, y cómo encaja todo**. Está pensado para que lo leas a tu ritmo, lo subrayes, y vuelvas a partes específicas cuando lo necesites. No es la documentación técnica del entregable — es tu manual personal de aprendizaje.

Índice

1. [El problema en lenguaje sencillo](#)
 2. [La idea general de la solución](#)
 3. [Conceptos previos que vas a necesitar](#)
 4. [Recorrido del repositorio](#)
 5. [Paso 1 — Setup del proyecto](#)
 6. [Paso 2 — EDA \(entender los datos\)](#)
 7. [Paso 3 — Feature engineering causal](#)
 8. [Paso 4 — Segmentación de clientes](#)
 9. [Paso 5 — Entrenamiento del modelo](#)
 10. [Paso 6 — Explicabilidad](#)
 11. [Paso 7 — Scoring batch + Parquet](#)
 12. [Paso 8 — Capa de storage cloud-agnóstica](#)
 13. [Paso 9 — El agente conversacional](#)
 14. [Paso 10 — La UI con Streamlit](#)
 15. [Paso 11 — Despliegue en la nube](#)
 16. [Cómo se conecta todo \(visión final\)](#)
 17. [Glosario rápido](#)
 18. [Preguntas frecuentes que probablemente tendrás](#)
-

1. El problema en lenguaje sencillo

Ferreycorp tiene un dataset con **visitas de clientes a una tienda durante 2 años**. Cada fila es una visita: el cliente entró ese día, vio precios y promociones de 5 marcas, y a veces compró, a veces no.

Te piden dos cosas:

1. **Un modelo predictivo:** dado un cliente y un día, ¿cuál es la probabilidad de que compre?
2. **Un agente conversacional:** que el equipo comercial pueda preguntarle cosas como *"dame los 100 clientes con mayor probabilidad de comprar que sean mujeres mayores de 40"* y obtener la respuesta sin pedirle ayuda al equipo de datos.

Analogía: imagínate que eres el dueño de una tienda con 500 clientes recurrentes. El modelo es como un **vendedor experto** que mira la ficha de cada cliente y te dice "este probablemente compre hoy, este no". El agente es como tener un **asistente** al que le hablas en español y te trae la respuesta del vendedor experto.

2. La idea general de la solución

La construimos en dos grandes capas, conectadas por un archivo:

[Datos crudos] ■ ■■■■ Capa 1 (offline, una sola vez al día/semana): ■ ■■■■ Modelo de ML que predice probabilidad por (cliente, día) ■ y guarda los resultados en un archivo Parquet ■ ■■■■ Capa 2 (online, en vivo): ■■■■ Agente IA + UI que lee ese Parquet y responde preguntas

¿Por qué dividirlo así? Porque entrenar un modelo y predecir 58 mil filas es lento (segundos a minutos). Hacerlo cada vez que un usuario hace una pregunta sería terrible. En cambio, **lo hacemos una vez** y guardamos el resultado. Las preguntas del usuario se responden en milisegundos consultando ese resultado.

Esto es un patrón estándar en producción y se llama **batch scoring + serving layer**.

3. Conceptos previos que vas a necesitar

Si alguna palabra no te suena, ven a esta sección.

3.1 Modelo predictivo

Una función matemática $f(\text{features}) \rightarrow \text{probabilidad}$ aprendida a partir de ejemplos pasados. Por dentro tiene parámetros (pesos, árboles, etc.) que se ajustan durante el "entrenamiento". Una vez entrenado, ante un cliente nuevo, le aplicas las features y te devuelve un número entre 0 y 1.

3.2 Features (variables predictoras)

Las "señales" numéricas/categóricas que le pasas al modelo. Ejemplo: edad, ingreso, cuántas compras hizo antes, etc. **El 80% del éxito de un modelo está en las features**, no en qué algoritmo elijas.

3.3 Target (variable objetivo)

Lo que quieres predecir. Aquí: `incidencia_compra` (0 o 1).

3.4 Entrenamiento, validación, test

Tres trozos del mismo dataset: - **Train:** el modelo "aprende" con estos datos. - **Validación:** usas estos datos para elegir hiperparámetros (sin que el modelo los vea durante el aprendizaje). - **Test:** la prueba final. Solo lo tocas al final, es la "estimación honesta" de qué tan bueno es el modelo en datos que nunca vio.

3.5 Data leakage

Cuando "se cuela" información del futuro en las features. Ejemplo de leakage que **NO** hicimos: usar la edad promedio de los clientes que compraron como feature → eso ya contiene info del target. Otro: si tu modelo predice si una visita termina en compra y le pasas como feature `cantidad`, **estás trampeando** porque `cantidad>0` solo existe si compró.

3.6 Probabilidad calibrada

Un modelo dice "70% probabilidad de comprar". Si tomas todas las visitas a las que les dijo 70% y mides cuántas realmente compraron, debería ser ~70%. Si dice 70% pero solo compran 30%, el modelo está **mal calibrado**.

3.7 LLM (Large Language Model)

GPT, Claude, Gemini. Modelos masivos que entienden y generan lenguaje. Aquí usamos Claude de Anthropic.

3.8 Tool use (uso de herramientas)

Capacidad del LLM de **llamar funciones programáticas** que tú defines. En vez de inventar la respuesta, el LLM dice "para responder esto necesito llamar la función `query_predictions` con estos argumentos", tu código la ejecuta, le devuelve el resultado al LLM, y el LLM redacta la respuesta final con datos reales.

3.9 Parquet

Un formato de archivo para guardar tablas. Como un CSV pero **mucho más eficiente** (10× más pequeño y rápido de leer). Lo usan todas las herramientas modernas de datos.

3.10 DuckDB

Una base de datos analítica **embebida** (no necesita servidor). Es como tener BigQuery o Athena dentro de tu programa Python. Lee Parquet directamente desde disco o desde la nube.

4. Recorrido del repositorio

```
ferreycorp_caso/ ■■■ data/ ■ ■■■ raw/ ← CSV original tal cual lo recibimos ■ ■■■
processed/ ← Datos transformados (con features) ■ ■■■ predictions/ ← Lo que produce el
modelo (consumido por el agente) ■■■ docs/ ← Toda la documentación (EDA, técnico, esta
guía) ■■■ deploy/ ← Specs para desplegar en cada nube ■■■ docker/Dockerfile ← "Receta"
para empaquetar la app ■■■ models/ ← Modelos entrenados serializados a disco ■■■ src/ ■
■■■ config.py ← Lectura de variables de entorno ■ ■■■ eda.py ← Análisis exploratorio ■
■■■ features.py ← Construcción de features ■ ■■■ model/ ■ ■ ■■■ train.py ←
Entrenamiento ■ ■ ■■■ explain.py ← SHAP, lift curve, calibración ■ ■ ■■■ score.py ←
Genera predicciones para todas las filas ■ ■■■ storage/ ■ ■ ■■■ object_storage.py ←
Abstracción cloud-agnóstica ■ ■■■ agent/ ■ ■ ■■■ tools.py ← Funciones que el LLM puede
llamar ■ ■ ■■■ agent.py ← Lógica del agente (Claude + tool use) ■ ■ ■■■ smoke_test.py
```

```
← Prueba rápida del agente ■ ■■■■ utils/io.py ← Cargador de datos ■ ■■■■ app.py ← UI
Streamlit ■■■■ scripts/build_pdfs.py ← Genera los PDFs entregables ■■■■ requirements.txt
← Dependencias Python ■■■■ .env.example ← Plantilla de variables de entorno ■■■■
README.md
```

Regla de oro: cada carpeta tiene un propósito único. `data/` no se commitea (datos pueden cambiar), `src/` es lo único que ejecutas, `docs/` es para humanos, `deploy/` es para máquinas.

Paso 1 — Setup del proyecto

Qué hicimos

Creamos un **entorno aislado** (`.venv/`) e instalamos las librerías. La idea de un `venv`: tu Mac puede tener Python 3.12 con cosas instaladas; este proyecto usa el SUYO propio para no contaminar.

Archivos involucrados

`requirements.txt` — Lista de todas las librerías y sus versiones exactas. Esto garantiza que en otra computadora se instalen las MISMAS versiones (reproducibilidad). Las más importantes:

- `pandas`, `numpy` → manipular tablas y arrays
- `scikit-learn` → utilidades de ML (split, scaling, K-Means, métricas)
- `lightgbm` → el modelo principal
- `optuna` → buscar hiperparámetros automáticamente
- `shap` → explicabilidad
- `duckdb`, `pyarrow` → leer/escribir Parquet rápido
- `anthropic` → cliente oficial de Claude
- `streamlit` → para la UI
- `boto3`, `s3fs`, `gcsfs` → clientes de las nubes

`.env.example` — Plantilla de variables de entorno. Cuando despliegues, NO escribes tu API key en el código (eso se filtra a Git y a cualquiera). Las pones aquí como variables de entorno.

`src/config.py` — Un archivo Python que lee el `.env` y expone una variable `config` que el resto del código importa. Si quieres cambiar el modelo de Claude, lo cambias en `.env`, no en código.

`docker/Dockerfile` — La "receta" para construir un contenedor. Dice "parte de una imagen Python 3.11, copia mis archivos, instala mis requirements, arranca Streamlit en el puerto 8501". Esto es lo que hace que la app sea **portable a cualquier nube**.

Concepto clave a entender: separación de configuración y código

Toda la lógica está en `src/`. La configuración (qué bucket usar, qué API key, qué nube) está en `.env`.

Cambiar de DigitalOcean a AWS no requiere tocar el código — solo cambias el `.env`. Esto se llama **12-factor app** y es el estándar moderno.

Paso 2 — EDA (entender los datos)

Qué es el EDA

Análisis Exploratorio de Datos. Antes de entrenar nada, tienes que ENTENDER los datos: qué columnas hay, cómo se distribuyen, hay nulos, hay outliers, qué relaciones existen, cuál es el balance del target.

Si te saltas el EDA, vas a tomar malas decisiones de modelado más adelante. Es como construir una casa sin medir el terreno.

Lo que hicimos en `src/eda.py`

Es un script que carga el CSV y genera **11 figuras** + un reporte en markdown. El reporte contesta:

1. **¿Cuántas filas y columnas?** 58,693 visitas × 23 columnas (después de quitar `tamano_ciudad`).
2. **¿Cuántos clientes únicos?** 500 — pocos, pero con muchas visitas cada uno (~117 promedio).
3. **¿Cuál es el target?** `incidencia_compra` con 25% de positivos. Esto es desbalance moderado, no severo. **No** necesitamos técnicas agresivas como SMOTE.
4. **¿Hay nulos?** Cero. Suerte ahí.
5. **¿Cómo se ven los precios?** Fluctúan en el tiempo → señal explotable.
6. **¿Las promos mueven la aguja?** Sí: 26.9% compran con promo vs 21.8% sin → +23% de lift. **Crítico.**
7. **¿Los clientes son leales a marcas?** **89% concentran >50% de sus compras en una sola marca.** Fuerte señal.
8. **¿Hay correlaciones brutales?** No, ninguna feature por sí sola explica el target. Eso significa que hay **interacciones** (cliente × precio × promo) y para eso son perfectos los modelos basados en árboles.

Por qué esto importa para el modelo

Cada hallazgo del EDA se convirtió en una **decisión de modelado**: - Sin nulos → no necesitamos imputación. - Promos suben el buy rate → tenemos que crear feature de "sensibilidad personal a promo". - Lealtad concentrada → feature de "% histórico por marca". - Precios fluctuantes → feature de "precio relativo al histórico".

Cómo correrlo

```
source .venv/bin/activate python -m src.eda
```

Genera: `docs/eda/REPORTE_EDA.md` y 11 PNGs en `docs/eda/figures/`.

Paso 3 — Feature engineering causal

Qué es feature engineering

Tomar el dato crudo y **construir nuevas variables que ayuden al modelo**. Aquí pasamos de 23 columnas crudas a **43 features**.

Qué quiere decir "causal"

Solo usar info de días ANTERIORES al día que estamos prediciendo. Si yo predigo si comprará el día 100, las features solo pueden mirar los días 1–99 de ese cliente.

¿Por qué? Porque en producción, cuando vayamos a predecir el "día de mañana", literalmente NO TENEMOS los datos de mañana. Si el modelo aprendió a usar info futura durante el entrenamiento, va a fallar miserablemente en producción. Esto es **data leakage** y es el error #1 en proyectos de ML.

Las familias de features que construimos en `src/features.py`

a) RFM (Recency, Frequency, Monetary)

Concepto clásico de marketing. Para cada visita: - **Recency**: días desde la última compra del cliente - **Frequency**: cuántas visitas previas tuvo / cuántas compras - **Monetary**: cantidad acumulada / promedio por compra

b) Lealtad de marca

5 columnas (`loyalty_b1` a `loyalty_b5`): % **histórico** de las compras de ese cliente que fueron de cada marca. Si compraste 8 veces marca 5 y 2 veces marca 2 (de 10 totales), entonces `loyalty_b5=0.8`, `loyalty_b2=0.2`, resto 0.

Más una columna `loyal_brand_id` que dice cuál es su marca dominante.

c) Sensibilidad personal a promo

- `buy_rate_with_promo`: % de visitas previas con al menos una promo en las que compró
- `buy_rate_no_promo`: % cuando no había promo
- `promo_uplift`: la diferencia. Si es alto, este cliente es **muy sensible a promos**.

d) Precio relativo

Para cada marca: `precio_de_hoy / promedio_histórico_de_la_marca`. Si está negativo, el precio bajó respecto al promedio.

e) Demografía

Edad, ingreso, género, etc. — directas del dataset.

Cómo se calcula sin loops (eficiencia)

Hacer esto con un `for cliente in clientes`: y un `for visita in visitas`: sería lentísimo. En su lugar usamos **groupby + cumsum + shift**:

```
# Cuenta acumulativa de compras por cliente, EXCLUYENDO la visita actual
df['prior_purchases'] = df.groupby('id')['incidencia_compra'].cumsum() -
df['incidencia_compra']
```

`cumsum` calcula la suma corriente. Restarle el valor actual te da "todo lo de antes". Esa simple resta es el truco para mantener todo causal y rápido.

Validamos que no hay leakage

Después de calcular las features, corrimos checks: - En la primera visita de cada cliente, todas las `prior_*` deben ser 0 → ■ - Las loyalties deben sumar 1 cuando hay compras previas, 0 cuando no → ■

Lección clave: en proyectos reales, escribir tests para tus features es tan importante como escribir tests para el código.

Cómo correrlo

```
python -m src.features
```

Genera `data/processed/features.parquet` (43 features × 58,693 filas).

Paso 4 — Segmentación de clientes

Qué es clustering / K-Means

Agrupar clientes que se "parecen" entre sí, sin tener etiquetas previas. **K-Means** es el algoritmo más común: tú le dices "quiero 4 grupos" y él encuentra los centros que minimizan la distancia promedio.

Analogía: tienes 500 personas en una sala. Pones 4 marcadores en el suelo. Cada persona camina al marcador más cercano. Mueves los marcadores al centro de su grupo. Repites. Eventualmente los marcadores no se mueven más → tienes 4 clusters.

Por qué lo hicimos

No para alimentar el modelo — sino para que el agente pueda filtrar por segmento. Le da al usuario un lenguaje natural ("Leales premium", "Cazadores de oferta") en vez de números abstractos.

Cómo lo hicimos en `src/features.py`

1. Para cada cliente, agregamos: tasa histórica de compra, # compras totales, ticket promedio, edad, ingreso.
2. Estandarizamos esas variables (`StandardScaler`) para que ninguna domine por su escala.
3. Corremos K-Means con `k=4`.
4. Etiquetamos los clusters con nombres legibles según sus características (función `label_clusters`):
5. **Leales premium** (47 clientes, 60% buy rate, ingreso alto)
6. **Cazadores de oferta** (45, 29% buy rate)
7. **Compradores moderados** (139, 22%)
8. **Visitantes ocasionales** (269, 18%)

¿Por qué no usamos el cluster como feature del modelo?

Porque para asignar el cluster usamos información de TODOS los días del cliente, incluyendo días posteriores al que estás prediciendo → leakage. Si quisiéramos usarlo como feature, tendríamos que recalcular el cluster con info hasta el día anterior, lo cual complica las cosas. Y como las features causales (`prior_buy_rate`, `loyalty_*`) ya capturan esto, no aporta nada nuevo.

Esta decisión es una **señal de madurez de ingeniería de ML**: distinguir entre "feature de modelo" y "atributo descriptivo para el negocio".

Paso 5 — Entrenamiento del modelo

Archivo: `src/model/train.py`.

Decisión 1: split temporal

Los datos van del día 1 al 730. **No** podemos hacer un split aleatorio (típico en ML) porque generaría leakage: el modelo vería días futuros del mismo cliente y aprendería patrones del futuro.

Hacemos un split por días: - Train: días 1–509 (~70%) - Validación: 510–619 (~15%) - Test: 620–730 (~15%)

Lección: cualquier vez que tu dato tenga orden temporal, splittea por tiempo, nunca al azar.

Decisión 2: dos modelos (baseline + final)

Baseline — Logistic Regression

Es el modelo lineal más simple para clasificación binaria. Asume que la probabilidad de comprar es:

$$P(\text{compra}) = \text{sigmoid}(w_1 \cdot \text{feature1} + w_2 \cdot \text{feature2} + \dots + b)$$

Aprende los pesos w que mejor separen 0s de 1s.

¿Por qué un baseline? Porque siempre quieres saber qué tan bueno es lo "simple". Si tu modelo complejo solo gana 1 punto al baseline, no vale la pena la complejidad.

LightGBM — el modelo final

Es un **Gradient Boosting Decision Tree (GBDT)**. Conceptualmente: 1. Empieza con una predicción base (ej: la media del target). 2. Construye un árbol pequeñito que predice el ERROR del paso 1. 3. Suma ese árbol a la predicción → reduce el error. 4. Repite cientos de veces.

Cada árbol es chiquito, pero la SUMA de cientos captura interacciones complejas. LightGBM es una implementación particularmente rápida y eficiente de esto.

Por qué LightGBM aquí: - Maneja categóricas natively (no necesitas one-hot encoding). - Captura interacciones automáticamente (cliente × precio × promo). - Es rápido en CPU. - Tiene una historia probada

de ganar competencias de Kaggle en datos tabulares.

Decisión 3: hyperparameter tuning con Optuna

LightGBM tiene ~10 hiperparámetros importantes (cuántos árboles, qué profundidad, cuánta regularización, etc.). Probarlos a mano es tedioso.

Optuna es una librería que automatiza esto. Le defines: - Un espacio de búsqueda (rangos para cada hiperparámetro) - Una métrica a optimizar (AUC en validación) - Cuántos intentos hacer (30 trials)

Optuna usa un algoritmo bayesiano: cada nuevo intento aprende de los anteriores y va a zonas prometedoras. Mucho más eficiente que un grid search ciego.

Decisión 4: las métricas

No nos importa solo "accuracy". Para propensión: - **AUC-ROC** (0.684): qué tan bien el modelo ORDENA. Si tomas dos clientes al azar (uno comprador, uno no), ¿con qué frecuencia el modelo le da más score al comprador? AUC=0.5 es random, AUC=1.0 es perfecto. - **PR-AUC**: similar pero más sensible a la clase minoritaria. - **Brier score**: qué tan calibradas están las probabilidades. - **Lift @ top-K**: si tomas el top-K% por score, ¿cuántas veces más compradores tiene que la población general? **Esta es la métrica de negocio**: si tu lift @ top-10% = 2.4, llamando solo al 10% top capturas 2.4× más compras que llamando al azar.

Cómo correrlo

```
python -m src.model.train
```

Tarda ~1-2 minutos. Genera: - `models/lgbm_propensity.txt` — el árbol serializado - `models/logreg_propensity.joblib` — el baseline - `docs/metrics/model_metrics.json` — todas las métricas

Paso 6 — Explicabilidad

Archivo: `src/model/explain.py`.

Qué es SHAP (en cristiano)

SHapley Additive exPlanations. Te dice, **para cada predicción individual**, cuánto contribuyó cada feature a subir o bajar el score, partiendo del promedio.

Analogía: imagina que el score es un partido de fútbol y arranca empatado en 0.25 (la tasa base). Cada feature es un jugador que mete o evita goles. SHAP te dice "prior_buy_rate metió +0.20, recency_days metió -0.05, etc.". Suma todo y te da el score final del cliente.

Por qué importa

El negocio te va a preguntar "¿por qué este cliente tiene score alto?". No basta con decir "porque el modelo lo dijo". Con SHAP puedes decir "porque su tasa histórica de compra es 50% (vs 25% promedio) y porque hoy hay promo en su marca leal".

Lo que generamos

- **shap_importance.png**: top 20 features ordenadas por impacto promedio (en valor absoluto).
- **shap_beeswarm.png**: nube de puntos con cada feature, donde se ve si valores altos suben o bajan el score.

Curva de Lift (cumulative gain)

Otra figura crítica. En el eje X: % de clientes ordenados por score. En Y: % de compras capturadas. **La línea diagonal es random**, la nuestra está bien encima → el modelo separa.

Visualmente le explicas al gerente: "si llamas al 10% top capturas el 24% de las compras; si llamas al 20% top capturas el 38%".

Curva de calibración

Cruza score predicho con tasa real observada. Si el modelo dice 50% pero solo compran 20%, está roto. La nuestra está bien alineada con la diagonal.

Cómo correrlo

```
python -m src.model.explain
```

Paso 7 — Scoring batch + Parquet

Archivo: `src/model/score.py`.

Qué hace

Recorre las 58,693 filas, le pide al LightGBM que prediga la probabilidad para cada una, y junta el resultado con info útil (demografía, segmento, decil) en una **tabla de predicciones**. La guarda como Parquet.

Por qué Parquet y no CSV

- Parquet es **columnar**: puedes leer solo las columnas que necesitas (rápido).
- Está **comprimido**: 5-10× más pequeño.
- Tiene **tipos**: no pierdes que `id` es int o que `score` es float.
- Lo soportan **todas** las herramientas modernas (DuckDB, Spark, BigQuery, pandas, polars).

Estructura de la tabla

```
id | dia_visita | edad | ingreso | ... | score_compra | decile | cluster_label
```

Cada fila es una predicción para un (cliente, día). El agente la consulta para responder preguntas.

Por qué hacerlo en batch (en vez de live)

Si cada vez que un usuario hace una pregunta tuviéramos que cargar el modelo y predecir 58k filas, cada respuesta tardaría minutos. En cambio:

1. **Una vez** (offline) corremos `score.py` y guardamos las predicciones.
2. El agente lee la tabla con DuckDB (sub-segundo).

Cuando los datos cambien, simplemente vuelves a correr `score.py` (semanal, diariamente, cuando llega data nueva).

Cómo correrlo

```
python -m src.model.score
```

Paso 8 — Capa de storage cloud-agnóstica

Archivo: `src/storage/object_storage.py`.

El problema

Si subes el Parquet a DigitalOcean Spaces, lees con `boto3` apuntando a un endpoint específico. Si lo subes a AWS S3, igual pero sin endpoint custom. Si a Google Cloud Storage, usas otra librería (`gcsfs`).

Si escribes el código asumiendo "es S3 de AWS", luego migrar a GCP es rehacer todo.

La solución: una clase `objectStorage` que abstrae las 3 nubes

Tiene 4 métodos públicos: - `write_parquet(df, key)` → escribe un Parquet donde sea - `read_parquet(key)` → lee un Parquet de donde sea - `get_duckdb_uri(key)` → da la URL que DuckDB puede usar - `configure_duckdb(conn)` → inyecta credenciales en DuckDB

Por dentro mira `config.cloud_provider` y elige la implementación correcta. **El resto del código nunca sabe en qué nube está.**

Por qué este patrón es importante

Cuando tu equipo de infra dice "vamos a migrar a GCP el próximo trimestre", no tienes que tocar `score.py`, `agent.py`, ni `app.py`. Solo cambias variables en `.env`. Esto se llama **dependency inversion**.

El truco de DuckDB con S3

DuckDB tiene una extensión `httpfs` que sabe leer S3 nativamente. Como Spaces y GCS son **S3-compatibles** (vía HMAC keys), puedes apuntar DuckDB a cualquiera de las tres con la misma sintaxis:

```
SELECT * FROM 's3://my-bucket/predictions.parquet'
```

Esto significa que el agente no cambia su código según la nube — solo cambia el endpoint que DuckDB usa.

Paso 9 — El agente conversacional

Aquí está la parte más interesante. Archivos: `src/agent/tools.py` y `src/agent/agent.py`.

El reto

El usuario escribe en español: *"dame los 50 clientes top que sean Leales premium con ingreso > 150k"*. Necesitamos que esto se traduzca a una consulta sobre el Parquet.

Las dos opciones que consideramos

Opción A — Text-to-SQL

Le das al LLM la estructura de la tabla y le pides que escriba SQL.

Pros: muy flexible, el LLM puede hacer cualquier consulta. **Contras:** - Si el LLM se equivoca con un nombre de columna, el SQL revienta. - El LLM podría escribir SQL malicioso o pesado (`DELETE`, `JOIN` cartesiano, etc.). - Difícil de testear y predecir.

Opción B — Tool use estructurado (lo que elegimos)

Defines funciones predefinidas y el LLM elige cuál llamar y con qué parámetros JSON.

Pros: - El LLM solo puede llamar funciones que existen. - Los parámetros se validan contra una whitelist antes de tocar SQL. - Predecible, testeable, seguro. **Contras:** menos flexible que SQL libre — pero para este caso de uso es suficiente.

Cómo funcionan las tools

Definimos 3 funciones en `src/agent/tools.py`:

1. `query_predictions(filters, order_by, limit, columns)`

"Dame filas individuales que cumplan estos filtros". El LLM emite algo como:

```
{ "filters": [ {"column": "cluster_label", "operator": "=", "value": "Leales premium"}, {"column": "ingreso_anual", "operator": ">", "value": 150000} ], "order_by": [{"column": "score_compra", "direction": "desc"}], "limit": 50 }
```

Nosotros: 1. **Validamos** que las columnas existan en una whitelist. 2. **Validamos** que los operadores estén permitidos. 3. **Construimos SQL parametrizado** (con ? y separamos los valores → previene injection). 4. Ejecutamos en DuckDB sobre el Parquet.

2. `aggregate_predictions(group_by, aggregations, ...)`

Para preguntas tipo "score promedio por segmento". Mismo patrón.

3. `schema_info()`

Devuelve la lista de columnas disponibles. Si el LLM duda, llama esto primero.

El loop del agente (`src/agent/agent.py`)

Cuando llega una pregunta del usuario, hacemos un **bucle de tool use**:

1. Mandamos al LLM: [system prompt + historia + pregunta + tools disponibles] 2. El LLM responde con uno de dos cosas: a) "stop_reason: end_turn" → hay un mensaje final, terminamos. b) "stop_reason: tool_use" → quiere llamar herramientas. Le ejecutamos las herramientas, pegamos los resultados como mensaje de "user" (rol tool_result), y volvemos al paso 1. 3. Ponemos un límite de 6 iteraciones por seguridad.

Es decir, en una sola pregunta el LLM puede hacer 2-3 llamadas a tools (ej: primero `schema_info`, luego `aggregate`, luego `query`) antes de redactar la respuesta final.

Por qué esto es elegante

- El LLM tiene **autonomía limitada**: puede explorar, agregar, filtrar, pero nunca puede ejecutar operaciones peligrosas.
- Cada tool call queda **registrada en el trace** → la UI puede mostrarte qué hizo el agente para llegar a su respuesta (transparencia).
- Si quieres añadir capacidades nuevas (ej: un what-if de pricing), solo agregas otra función a `tools.py` con su schema.

Paso 10 — La UI con Streamlit

Archivo: `src/app.py`.

Qué es Streamlit

Una librería que te deja convertir un script Python en una webapp con dos líneas de código. No necesitas saber HTML/CSS/JS. Para POCs es **brutalmente eficiente**.

Limitación: no es para apps de producción a escala (1000s de usuarios concurrentes). Pero para herramientas internas de equipos, es perfecto.

Las 3 vistas

■ Chat

- Sidebar con botones de preguntas de ejemplo.
- Caja de texto donde el usuario tipea.
- Cuando llega respuesta del agente, muestra el texto + un acordeón con las tool calls que hizo (transparencia: el negocio puede ver QUÉ consultó).

■ Dashboard

- Métricas globales (# predicciones, # clientes, score promedio).
- Distribución de scores (histograma).
- Score predicho vs real por decil (validación visual).
- Distribución por segmento.
- Filtros interactivos para explorar.

■ Modelo

- Métricas finales del modelo.
- Imágenes de SHAP, lift curve, calibración.

Cómo se conecta con el resto

- Carga las predicciones con `duckdb.sql(...).df()` directo del Parquet (rápido).
- Instancia un `PropensityAgent()` cuando hay API key.
- Cachea la conexión y los datos con `@st.cache_data` y `@st.cache_resource` (Streamlit reutiliza entre interacciones del usuario).

Cómo correrla

```
streamlit run src/app.py
```

Abre `http://localhost:8501`.

Paso 11 — Despliegue en la nube

El concepto: una imagen Docker, tres nubes

El `Dockerfile` en `docker/` define una imagen autocontenida con todo lo necesario. Cualquier servicio de cómputo en cualquier nube puede correrla.

DigitalOcean App Platform (`deploy/digitalocean/app.yaml`)

Es lo más simple. App Platform clona tu repo de GitHub, construye la imagen del Dockerfile, la corre, y te da un dominio público. Costo: ~\$12-25/mes.

AWS ECS Fargate (`deploy/aws/ecs-task.json`)

Más complejo pero estándar enterprise. Defines una "task definition" (cuánta CPU, RAM, qué imagen, qué variables de entorno, qué secretos), un "service" que la mantiene corriendo, un load balancer al frente. Más control, más complejidad.

GCP Cloud Run (deploy/gcp/cloudrun.yaml)

Similar a App Platform en filosofía. Empuja la imagen a Container Registry, defines el spec YAML, `gcloud run deploy`. Cobra por uso (tiempo de CPU efectivo).

Lo crítico

El código fuente es idéntico en las tres. Cambia: - El `CLOUD_PROVIDER` env var. - Las credenciales de storage. - El comando para desplegar (`doctl apps create` vs `aws ecs ...` vs `gcloud run ...`).

Esto es lo que querías y por qué insistimos en la abstracción de storage.

Cómo se conecta todo (visión final)

Ahora con todo el contexto, vuelve a leer este flujo:

1. USUARIO: tipea pregunta en Streamlit
 2. STREAMLIT: pasa la pregunta al PropensityAgent
 3. AGENT: la manda a Claude con el system prompt y los tools
 4. CLAUDE: decide llamar `query_predictions` con filtros JSON
 5. tools.py: valida, construye SQL parametrizado
 6. DuckDB: ejecuta SQL sobre `predictions.parquet`
 7. PARQUET: vive en local / Spaces / S3 / GCS según `.env`
 8. RESULTADO: viaja de vuelta hasta Claude
 9. CLAUDE: redacta respuesta en español usando los datos reales
 10. STREAMLIT: muestra la respuesta + el trace

Y antes de todo eso, el "trabajo offline":

- A. CSV crudo B. EDA → entendimiento C. `features.py` → 43 features causales + segmentos D. `train.py` → LightGBM tuneado E. `explain.py` → SHAP + lift + calibración (para confianza del negocio) F. `score.py` → `predictions.parquet` (lo que consume el agente)

Si actualizas datos: corres D y F otra vez. **A, B, C** solo cuando cambia la lógica.

Glosario rápido

Término	Significado breve
EDA	Análisis exploratorio: entender los datos antes de modelar
Feature	Variable de entrada al modelo
Target	Variable que queremos predecir

Leakage	Cuando info del futuro se cuela en las features → modelo trampea
Causal feature	Feature calculada solo con info anterior al evento a predecir
Train/Val/Test	Tres splits del dataset, cada uno con un rol distinto
Split temporal	Splittear por fecha en vez de aleatoriamente
Logistic Regression	Modelo lineal para clasificación binaria
GBDT	Gradient Boosting Decision Trees (LightGBM, XGBoost, CatBoost)
Hiperparámetros	Parámetros que tú eliges, no el modelo (ej: # de árboles)
Optuna	Librería que busca hiperparámetros óptimos automáticamente
AUC-ROC	Métrica de qué tan bien ORDENA el modelo (0.5 random, 1.0 perfecto)
PR-AUC	Como AUC pero más sensible a la clase minoritaria
Lift @ K	Cuántas veces más positivos hay en el top-K vs el promedio
Brier score	Métrica de qué tan calibradas están las probabilidades
Calibración	Si el modelo dice 70%, debería acertar el ~70% de las veces
SHAP	Técnica de explicabilidad: cuánto contribuyó cada feature a una predicción
K-Means	Algoritmo de clustering: agrupa puntos similares sin etiquetas
RFM	Recency / Frequency / Monetary — features clásicas de marketing
Parquet	Formato de archivo columnar, comprimido, tipado
DuckDB	Base de datos analítica embebida (sin servidor)
Batch scoring	Predecir todas las filas de una sola vez y guardar el resultado
LLM	Large Language Model (Claude, GPT, etc.)
Tool use	Cuando el LLM llama funciones programáticas en vez de inventar

System prompt	Instrucciones iniciales que le das al LLM (rol, reglas)
Whitelist	Lista cerrada de cosas permitidas (vs blacklist = lista de prohibidas)
SQL parametrizado	SQL con placeholders (?) para prevenir inyección
Streamlit	Librería Python para hacer webapps rápidas
Docker	Tecnología de containers: empaqueta tu app con todo lo que necesita
Container	Una "cápsula" portable que corre igual en cualquier máquina
Cloud-agnóstico	Diseñado para correr en cualquier nube sin cambiar código
fsspec / boto3 / gcsfs	Librerías para hablar con storage en la nube
HMAC keys	Credenciales tipo S3 que también funcionan con DO Spaces y GCS
12-factor app	Conjunto de buenas prácticas para apps cloud-native

Preguntas frecuentes que probablemente tendrás

"¿Por qué usamos LightGBM y no una red neuronal?"

Para datos tabulares (filas con features estructuradas), GBDT casi siempre gana o empata a redes neuronales. Las redes brillan en imágenes, audio, texto crudo — no en tablas. Además LightGBM es 100× más rápido de entrenar.

"¿El modelo es bueno con AUC 0.68?"

Para predicción de propensión por visita en retail, sí. Estamos en el rango típico (0.65-0.75). Lo que más importa para el negocio es el **lift @ top-K = 2.4×** — eso significa que en producción el modelo Sí ayuda a priorizar campañas.

"¿Por qué K=4 en el clustering y no 3 o 5?"

Probamos visualmente y 4 dio segmentos interpretables. Hay técnicas como el "elbow method" o el silhouette score para elegir K más rigurosamente, pero como esto es solo para etiquetar (no input del modelo), 4 funciona y es legible.

"¿Por qué no entrenamos un modelo por marca también?"

Lo dejamos como extensión futura. El problema principal era propensión binaria. Un modelo multiclase de marca tendría clases muy desbalanceadas (Marca 3 = 5.7%) — requeriría tratamiento especial (focal loss,

class weights agresivos).

"¿Qué pasa cuando llegan datos nuevos?"

Re-corres `features.py`, `train.py`, `score.py` — en ese orden. En producción esto se automatiza con un orchestrator (Airflow, Prefect, Dagster, GitHub Actions cron) que corre semanalmente.

"¿El agente puede 'inventar' datos?"

No. La instrucción del system prompt es explícita: "no inventes números, siempre consulta primero". Y en la práctica, las preguntas típicas son sobre filtros y agregaciones que tienen UNA respuesta correcta calculable. Si el agente intentara inventar, el trace lo delata (no habría tool calls).

"¿Por qué DuckDB y no Postgres?"

DuckDB: - Embebido (sin levantar un servicio). - Optimizado para queries analíticas (OLAP, no OLTP). - Lee Parquet remoto directamente. - Cero costo de infra para POC.

Postgres tiene su lugar (transacciones, escrituras concurrentes, joins gigantes), pero para "leer un Parquet con filtros" es overkill.

"¿Cómo escalamos esto a 1M de clientes?"

- Storage: ya escala (S3/Spaces/GCS son ilimitados).
- Modelo: LightGBM entrena con millones de filas tranquilo (más RAM).
- Scoring: paralelizable — partir el dataset y correr en N workers.
- Agente: limitar tamaño del Parquet o particionar (uno por mes, por región).
- DuckDB: a 100M+ filas, podría conviene migrar a BigQuery/Athena. Pero hasta 50M filas, DuckDB sobre Parquet sigue siendo lo más simple y barato.

"¿Por qué no commiteamos los datos?"

- Los datasets pueden ser grandes (Git no maneja bien archivos > 100MB).
- Pueden contener datos sensibles.
- Cambian con el tiempo y eso ensucia la historia de Git. La regla: en `.gitignore` van datos, modelos serializados grandes, `.env`, `.venv`, etc.

"¿Por qué el Brier score baja con LightGBM?"

Brier mide error cuadrático en probabilidades. LightGBM con regularización adecuada (`lambda_l1`, `lambda_l2` que tuneamos con Optuna) genera probabilidades más cercanas a la realidad → menor Brier.

"¿Y si Claude está caído?"

La UI mostraría un error específico. La parte del modelo (Streamlit dashboard) sigue funcionando. **Diseño con desacoplamiento:** el dashboard no depende del agente para funcionar.

¿Te quedaste con dudas en alguna sección? Dímelas y te las explico con más detalle o agregó más analogías.